SMS Spam Detection with Grouped and External Evaluation

Kabiyev Altynbek
Astana IT University, Kazakhstan

Abstract

This study evaluates eleven sparse text classifiers on the historical English SMS Spam Collection, separating training search, validation threshold selection and final testing. Deduplication leaves 5,171 messages, with normalized groups kept together in holdouts and training folds. The selected word TF-IDF LinearSVC achieves spam F1 0.9531, with four false positives and eight false negatives among 1,034 test messages. Its conditional 95% group-bootstrap F1 interval is 0.9249 to 0.9769. Separator insertion at intensity 0.3 lowers mean recall from 0.9385 to 0.6035. Character features retain more recall at operating points that can increase false positives. An exploratory template split qualifies the historical comparison. We then evaluate the unchanged artifact on independent published SMS reports under a protocol fixed before prediction. Among 372 distinct English texts with explicit 2023–2024 years, it detects 176 and misses 196: recall 47.31%, with a conditional 95% interval of 42.32% to 52.28%. The pseudonymized positive-only sample cannot estimate modern false-positive rate or F1. High historical quality therefore does not establish reliable transfer to reported smishing.

Keywords: SMS spam, TF-IDF, grouped evaluation, external evaluation, character obfuscation.

1 Introduction

SMS filtering must detect unwanted messages without suppressing legitimate communication. Class imbalance makes accuracy an incomplete measure: on this test set, always predicting ham achieves 87.43% accuracy but detects no spam. We therefore report spam precision, recall and F1 together with false-positive and false-negative counts. We also separate clean-text quality from sensitivity to character edits that can disrupt a vocabulary-based classifier.

The contribution is an auditable comparison of representations and decision rules. Candidates share training folds and attack realizations. Model choice is frozen before test scoring, and validation chooses the threshold. A second grouping rule examines residual template overlap. An independent collection of reported SMS then tests transfer with the original model and threshold; these outcomes do not select or retrain the model.

2 Related work

The SMS Spam Collection accompanies Almeida, Hidalgo and Yamakami's benchmark research [1, 2]. Its historical sources limit deployment claims. ExplainableDetector studies transformer detection and explainability [3]; SpamDam investigates newer SMS, federated learning and adversaries [4]; COPS studies an on-device smishing and URL pipeline [5]. Their metrics are not directly comparable to our splits. Agarwal et al. release public smishing reports collected during 2017–2024 [9]. We use that independent artifact for a separate frozen-model evaluation, without claiming to reproduce or outperform those systems.

3 Data and evaluation protocol

The bundled corpus contains 5,574 labeled messages [1]. Input checks reject missing or blank training text, unsupported labels and conflicting labels within a group. Removing 403 exact duplicate rows leaves 4,518 ham and 653 spam messages. The remaining original strings are preserved as model inputs. Lowercasing and collapsing whitespace define 5,159 split groups; punctuation and Unicode remain in these keys. Message and group hashes provide an auditable membership record.

Table 1. Default protocol partitions after exact deduplication.

Partition	Ham	Spam	Total	Purpose
Train	2710	392	3102	Model search and fitting
Validation	904	131	1035	Threshold selection
Test	904	130	1034	Final reporting

Unique homogeneous groups are stratified by label into approximately 60% training, 20% validation and 20% test data, using seeds 42 and 43. Whole groups remain in one partition. Five shared StratifiedGroupKFold folds inside training use seed 42. Every cleaner, vocabulary and inverse-document-frequency transformation is fitted within its training fold. No validation or test text enters feature fitting. Grouping reduces a declared form of overlap; it does not prove sender, campaign or temporal independence.

4 Models and operating thresholds

The implementation uses scikit-learn pipelines [6]. Four original word-feature baselines are Multinomial Naive Bayes, Logistic Regression, linear-kernel SVC and Random Forest. Seven LinearSVC variants compare word and character representations under legacy cleanup, gentle cleanup, their word-plus-character combination, and character models with punctuation or currency symbols removed. Legacy cleanup lowercases text and retains only a-z, 0-9 and whitespace. Gentle cleanup lowercases and collapses whitespace without stripping symbols.

Word TF-IDF uses unigrams and bigrams; char_wb TF-IDF uses character 3- to 5-grams within word boundaries. Vocabulary budgets are 1,000 or 5,000 features; the combined representation divides the budget equally and normalizes the concatenation. Search uses alpha in {0.1, 1} for Naive Bayes; C in {0.5, 1} and class weights in {none, balanced} for linear models; and depth in {unlimited, 20} with minimum leaf size in {1, 2} for the 100-tree forest. There are 84 candidates and 420 fold fits.

Each variant retains its highest mean training-CV spam F1 configuration. The overall choice uses that same criterion, breaking model ties alphabetically. Best searched CV scores are selection scores, not independent estimates. Each chosen pipeline is refitted on training only. Its validation threshold maximizes recall subject to empirical FPR <= 1%, then prefers fewer false positives and a higher threshold. Decisions use score >= threshold. The model is not refitted afterward. Native default predictions are reported separately; the validation constraint does not guarantee future FPR [7].

5 Clean test results

Table 2. Default protocol. FP/FN and FPR use the validation threshold. *CV-selected model.

Variant	CV F1	Default F1	Tuned F1	FP/FN	FPR
NB word legacy	0.9099	0.9053	0.9213	7/13	0.77%
LR word legacy	0.9111	0.9035	0.9120	6/16	0.66%
SVC word legacy	0.9249	0.9091	0.9339	7/10	0.77%
RF word legacy	0.8707	0.8908	0.8855	16/14	1.77%
LinearSVC word legacy	0.9283	0.9323	0.9286	5/13	0.55%
LinearSVC char legacy	0.9407	0.9520	0.9164	19/4	2.10%
LinearSVC word gentle*	0.9460	0.9486	0.9531	4/8	0.44%
LinearSVC char gentle	0.9433	0.9690	0.9203	19/3	2.10%
LinearSVC word + char	0.9435	0.9393	0.9575	5/6	0.55%
LinearSVC char no punctuation	0.9409	0.9478	0.9615	5/5	0.55%
LinearSVC char no currency	0.9434	0.9690	0.9403	12/4	1.33%

Training CV selects word/gentle LinearSVC (C = 1, balanced class weights, vocabulary limit 5,000). At threshold -0.08620419778818611, test precision is 0.9683 and recall is 0.9385. The confusion counts are TN = 900, FP = 4, FN = 8 and TP = 122. Accuracy is 0.9884, ROC-AUC is 0.9957 and average precision is 0.9817. Native default predictions yield three false positives and ten false negatives, so tuning exchanges one additional false positive for two fewer missed spam messages.

Several alternatives have higher test F1 under particular decision rules. They do not replace the CV choice. Tuning reduces test F1 for several variants, including the character/gentle model: its recall increases, but 19 of 904 legitimate messages become false positives (2.10% FPR). This illustrates the difference between the validation objective and a guarantee on unseen messages. All eleven models are retained to make these tradeoffs visible.

6 Conditional uncertainty

For the fixed selected model, we sample whole test groups with replacement separately within ham and spam, preserving all messages in each selected group. We use 5,000 draws, seed 42, and the 2.5th and 97.5th percentiles of the resulting metric distribution [8]. The default protocol contains 1,032 test groups. The tuned F1 interval is 0.9249 to 0.9769; recall is 0.8931 to 0.9769 and FPR is 0.11% to 0.88%. Predictions reconstructed from the complete error ledger must match the saved confusion counts before resampling.

These intervals condition on the fitted model, decision rule, split and observed class/group composition. They exclude training and model-search variability, unknown campaign dependence and domain shift. With few errors, percentile intervals can be discrete or degenerate. They are approximate descriptive uncertainty estimates, not evidence that the same coverage holds in a new SMS population.

7 Character obfuscation experiments

Three nonadaptive scenarios modify raw spam before the fitted preprocessing pipeline. Leet replaces eligible ASCII letters using a fixed substitution map. Separator insertion places a period, hyphen or underscore between adjacent ASCII letters. Unicode substitution replaces eligible Latin letters with specified Cyrillic lookalikes. Intensity is the edit probability per eligible character or boundary, rather than the fraction of all characters changed. We use intensities 0.0 to 0.5 in steps of 0.1 and twenty seeds, 10042 to 10061.

Every model receives the same attacked messages for each scenario, intensity and seed. Ham remains unchanged, and clean validation thresholds stay fixed. Attack success rate (ASR) is the fraction of initially correctly detected spam that becomes ham after editing; its denominator excludes spam already missed on clean input. Twenty seeds measure randomness of these edits on the same test messages, not uncertainty across new training or test sets.

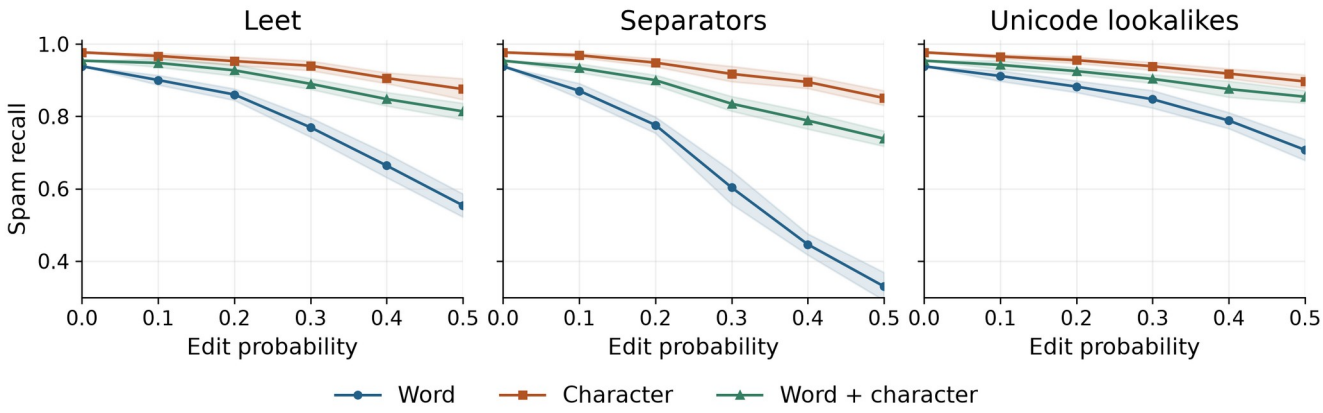


Figure 1. Default protocol, gentle cleanup and validation thresholds. Curves show mean spam recall; bands show one attack-seed standard deviation. All classifiers are LinearSVC. Different clean false-positive rates must be considered alongside recall.

Table 3. Mean recall (R) and ASR at intensity 0.3, with gentle cleanup and tuned thresholds.

Features	Clean FPR	Leet R / ASR	Separators R / ASR	Unicode R / ASR
Word	0.44%	0.770 / 0.182	0.603 / 0.358	0.847 / 0.098
Character	2.10%	0.940 / 0.039	0.917 / 0.061	0.938 / 0.040
Word + character	0.55%	0.890 / 0.070	0.835 / 0.128	0.903 / 0.055

The selected word model is most sensitive to separator insertion at intensity 0.3: mean recall drops to 0.6035 and ASR reaches 0.3582. Character features retain more recall across these scenarios, but their tuned clean FPR is 2.10%, compared with 0.44% for words. The combined model offers an intermediate measured tradeoff. These observations do not isolate a universal representation advantage: configurations and operating points differ. Human readability, meaning preservation, adaptive optimization and modifications to ham were not evaluated.

8 Sensitivity to message templates

Case and whitespace grouping can separate messages that differ only in a phone number or URL. A diagnostic rule masks URLs and digit runs in normalized messages longer than thirty characters. Short messages retain their normalized key to reduce accidental merging of unrelated short replies. The rule is stateless, uses no model scores and changes grouping only; classifiers still receive original strings. Conflicting labels within a template group are rejected.

The default split contains thirteen templates crossing partition boundaries, covering thirty-one messages; six test messages have a training counterpart under this rule. A separate complete experiment uses the 5,137 template groups for both holdout splitting and training CV. It produces 3,095 training, 1,040 validation and 1,036 test messages, including 904 ham and 132 spam in test. No template crosses partitions under the declared rule. This does not identify every semantic paraphrase or campaign.

Table 4. Template protocol, CV-selected character LinearSVC without punctuation.

Decision rule	Precision	Recall	F1	FP/FN	95% F1 interval
Native default	0.9839	0.9242	0.9531	2/10	0.9237 to 0.9769
Validation tuned	0.9185	0.9394	0.9288	11/8	0.8968 to 0.9586

Training CV selects the character model without punctuation in this experiment. Tuning raises recall slightly but increases false positives from two to eleven and lowers F1 from 0.9531 to 0.9288. Test FPR reaches 1.22% despite meeting the validation constraint. The conditional intervals use 1,028 template test groups. These findings retain the unfavorable outcome rather than choosing a more flattering threshold after evaluation.

The template analysis was introduced after inspecting the default experiment and is exploratory. The two protocols change the split, fitted models and selected configurations together. Their difference is not a causal estimate of leakage, and their F1 intervals are not a paired significance test. The default model remains the frozen choice for its original protocol.

9 Scope of the historical experiments

Both training protocols use one historical English corpus. Row order is not a timestamp, and the available labels do not establish independent senders or campaigns [1]. Related messages may remain after either grouping rule. Section 10 therefore tests the original model on a separate reported-SMS collection. Russian or Kazakh text, code-switching, email and URLs as separate objects remain unmeasured. A ham prediction does not establish that a link or sender is safe.

The external sample has no legitimate messages, so independent modern ham and a new spam holdout are still needed to assess the full error tradeoff. Source/date annotations and representative sampling also matter. Any defense developed after inspecting the reported failures needs new final evaluation data. Deployment additionally requires measurements of latency, memory, privacy behavior and false-positive cost; sparse features alone do not establish on-device suitability.

10 External evaluation on reported SMS

We use the CC BY 4.0 author artifact from Fishing for Smishing [9], pinned at commit a6175560b57387199871e51fbef6bc523d2516b4. It contains 33,869 pseudonymized reports with publisher-supplied text and annotations, partly extracted using language models. We use original English text only. A 2025 release does not imply 2025 message dates. Requiring one explicit 2023 or 2024 year yields 488 rows, mostly 2023; normalized deduplication leaves 372 texts in 371 template groups. The supplementary cohort contains 15,531 distinct English texts across all/unknown years. Neither cohort matches the entire historical corpus under exact, normalized or template keys; paraphrase and campaign independence are not established.

Source, membership, artifact hash and decision rules were fixed before external prediction (repository PR #8). No fitting or threshold selection occurs. All inputs, including OOV messages, remain in the denominator. Table 5 uses the published pseudonymized text; the two overlapping cohorts are not independent replications.

Table 5. Frozen model on positive reports. No external F1, precision, accuracy or FPR estimate.

Cohort	Decision	Detected / total	Missed	Recall	95% interval
2023–2024	Validation	176 / 372	196	47.31%	42.32–52.28%
2023–2024	Native	145 / 372	227	38.98%	33.96–43.97%
All English	Validation	7,491 / 15,531	8,040	48.23%	47.45–49.02%
All English	Native	6,109 / 15,531	9,422	39.33%	38.56–40.09%

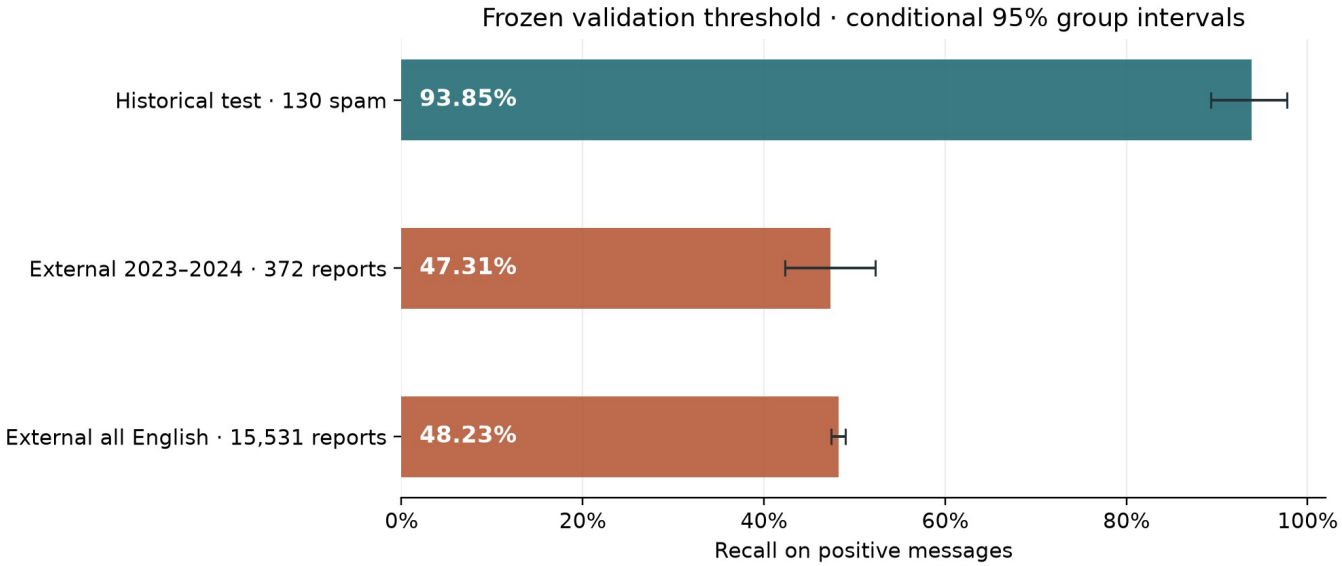


Figure 2. Same model and validation threshold. Intervals condition on observed groups; external cohorts overlap. Source, sampling, message type and pseudonymization differ, so the contrast is not a causal estimate of temporal degradation.

The primary result is 47.31% recall, with 196 misses. Removing the fixed publisher placeholder tokens gives 47.04% (175 detected), a 0.27 percentage-point change; it cannot recover unredacted originals. Intervals use 5,000 whole-template resamples, seed 42, and exclude training, annotation and sampling uncertainty. Category counts are descriptive. This positive-report convenience sample exposes limited transfer but cannot measure the cost of blocking legitimate SMS.

11 Reproducibility and conclusion

The repository supplies raw-data checks, split and CV manifests, all 84 search results, clean and validation metrics, attack seeds and hashes, complete error ledgers, interval reports and fitted-pipeline export. Dataset, training source and model artifacts have SHA-256 records. Both full protocols were independently checked against saved models, including all 7,920 attacked metric rows per run. The environment uses Python 3.12.14 and scikit-learn 1.9.1; exact dependencies and execution metadata are recorded with each experiment.

The README documents training and historical verification; docs/EXTERNAL_EVALUATION.md records the pinned external source, cohort construction and scoring commands. scripts/verify_external.py reconstructs all 31,806 score rows, 63,612 binary decisions, eight summaries, 72 category rows and intervals directly from the fitted pipeline. External verification requires the original hash-matched local artifact; binaries are not committed. Output directories must be unused, and retraining cannot silently replace the frozen model.

The word model combines high historical F1 with sensitivity to separator edits and low recall on independent reported SMS. Character features retain more recall under the synthetic edits, at operating points that can increase false positives. The external result leaves modern false-positive cost unresolved. Future development needs representative modern ham and new spam, with an untouched final holdout reserved before using these inspected failures to improve the model.

Code and experiment artifacts: https://github.com/altynbk/sms-spam-detection

References

[1] T. Almeida and J. Hidalgo. SMS Spam Collection. UCI Machine Learning Repository, 2012. Dataset DOI: 10.24432/C5CC84. https://archive.ics.uci.edu/dataset/228/sms+spam+collection

[2] T. A. Almeida, J. M. G. Hidalgo and A. Yamakami. Contributions to the study of SMS spam filtering: new collection and results. ACM Symposium on Document Engineering, 2011. Publication record linked from [1].

[3] M. A. Uddin et al. ExplainableDetector: Exploring Transformer-based Language Modeling Approach for SMS Spam Detection with Explainability Analysis. arXiv:2405.08026, 2024. https://arxiv.org/abs/2405.08026

[4] Y. Li, R. Zhang, W. Rong and X. Mi. SpamDam: Towards Privacy-Preserving and Adversary-Resistant SMS Spam Detection. arXiv:2404.09481, 2024. https://arxiv.org/abs/2404.09481

[5] Harichandana B S S et al. COPS: A Compact On-device Pipeline for real-time Smishing detection. IEEE CCNC, 2024. arXiv:2402.04173. https://arxiv.org/abs/2402.04173

[6] F. Pedregosa et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html

[7] Scikit-learn developers. Tuning the decision threshold for class prediction. User Guide, accessed 9 October 2026. https://scikit-learn.org/stable/modules/classification_threshold.html

[8] SciPy developers. scipy.stats.bootstrap. Reference documentation, accessed 9 October 2026. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.bootstrap.html

[9] S. Agarwal, A. Papasavva, G. Suarez-Tangil and M. Vasek. Fishing for Smishing: Understanding SMS Phishing Infrastructure and Strategies by Mining Public User Reports. ACM IMC, 2025. DOI: 10.1145/3730567.3764431. Dataset: https://github.com/reportsmishing/Smishing-Dataset-IMC25